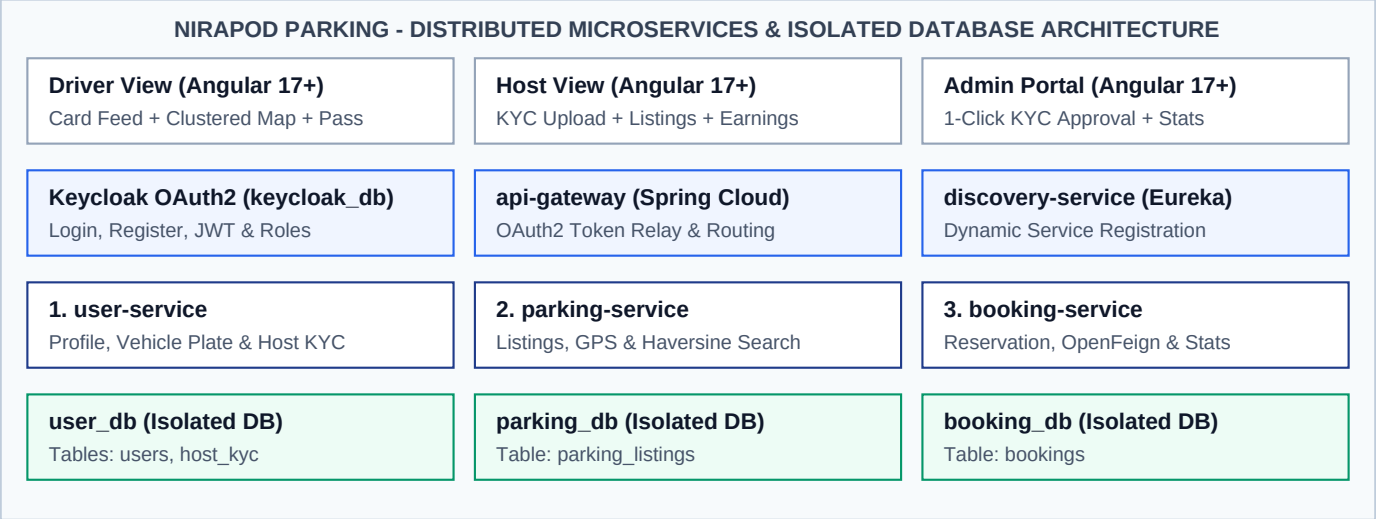
Nirapod Parking: Complete System Blueprint & Defense Specification

The Community-Driven Residential Parking Marketplace | Final Year Capstone Blueprint

Core Execution Strategy (MVP-First -> Incremental A+ Upgrades)

* **Stage 1 (Core Microservices + Minimal MVP):** Lay out the enterprise Spring Cloud Microservices architecture (api-gateway, discovery-service, user-service, parking-service, booking-service) with Database-per-Service isolation, secured by Keycloak (OAuth2 / OIDC), including Card-First search, online KYC, basic booking, and Host/Admin Dashboard Summary Cards (SUM/COUNT).
* **Stage 2 (Geospatial "Wow Factor"):** Plug in Haversine Proximity Search ("Find Near Me") and the Airbnb-style Split-View Leaflet Clustered Map (Leaflet.markercluster) using the GPS coordinates already captured in Stage 1.
* **Stage 3 (Lightweight Defense Polish):** Add a simple SQL overlap check (preventing double-booking) and a printable Digital Parking Pass (with Vehicle Plate matching).

1. System Architecture (Microservices + Database-per-Service + Keycloak OAuth2)



2. Database-per-Service Schema (3 Isolated Databases, 4 Core Tables)

Each microservice owns its own isolated database. There are no cross-database foreign keys; cross-service references (host_user_id, listing_id, driver_user_id) are stored as logical IDs and validated via OpenFeign REST calls.

A. user_db (Owned exclusively by user-service)

Table	Columns	Key Details & Purpose
users	id (PK), keycloak_id (UK), full_name, email, phone_number, vehicle_plate, is_verified_host, created_at	Stores unified Driver/Host profile synced from Keycloak JWT. is_verified_host becomes true after Admin KYC approval.
host_kyc	id (PK), user_id (FK -> users.id), nid_number, nid_photo_url, ownership_doc_url, slot_photo_url, status, admin_remarks	Stores Uber/Pathao-style Host KYC documents (NID, Holding Tax/Allotment Slip, Garage Bay Photo) and status (PENDING, APPROVED, REJECTED).

B. parking_db (Owned exclusively by parking-service)

Table	Columns	Key Details & Purpose
parking_listings	id (PK), host_user_id (Logical ID), title, area_name, full_address, latitude, longitude, vehicle_type, hourly_rate, available_from, available_until, has_cctv, has_guard, status	Every Host owns 100% of their listing row. Latitude & longitude are captured via 1-click Browser GPS in Stage 1 and reused for Haversine & Map Clustering in Stage 2.

C. booking_db (Owned exclusively by booking-service)

Table	Columns	Key Details & Purpose
bookings	id (PK), listing_id (Logical ID), host_user_id (Logical ID), driver_user_id (Logical ID), driver_vehicle_plate, start_time, end_time, total_amount, status, created_at	Stores reservations, calculates total_amount (Hours x Rate), and provides fast SUM/COUNT queries for Host earnings and booking statistics.

3. Stage 1 End-to-End Operational Workflow

Unified Dual-Role Concept (The "Mirpur-to-Banani Commuter" Model): Every user signs up once via Keycloak and starts as a Driver by default. If they also own a flat with an empty daytime garage, they click [Become a Host], upload their KYC proof, and once approved by the Admin, they can switch freely between Driver View and Host Dashboard using the same account.

Workflow Phase	Services Involved	Step-by-Step Execution
Flow 1: Sign-Up & Driver Profile	Keycloak -> api-gateway -> user-service (user_db)	1. User registers/logs in via Keycloak OAuth2 login page. 2. Completes initial profile with Phone Number and Vehicle Plate Number. 3. Ready to search and book parking as a Driver.
Flow 2: "Become a Host" & Admin KYC	user-service (user_db)	1. User clicks "Become a Host" and uploads NID photo, Holding Tax/Allotment Slip, and Garage Pillar Photo. 2. Admin reviews Pending KYC in Admin Portal and clicks [Approve]. 3. is_verified_host becomes true.
Flow 3: Creating a Parking Spot	parking-service (parking_db) <-> user-service	1. Host enters Title, Area, Address, Vehicle Type, Rate, and clicks [Capture Current GPS]. 2. parking-service calls user-service via OpenFeign to verify is_verified_host == true. 3. Saves listing in parking_db.
Flow 4: Searching, Booking & Navigating	parking-service <-> booking-service (booking_db)	1. Driver searches by Area + Vehicle Type in Card Feed and views Google Maps iframe preview. 2. Picks Start/End Time and confirms booking. 3. booking-service fetches rate via OpenFeign, saves booking, updates Host stats, and unlocks Google Maps navigation.

4. Three-Stage Incremental Build Roadmap

STAGE 1: Core Microservices Architecture + Minimal Functional MVP (Build First)

* **Infrastructure Setup:** Run Keycloak (keycloak_db), discovery-service (Eureka), and api-gateway (Spring Cloud Gateway).
* **user-service (user_db):** Sync logged-in Keycloak user, save Driver phone & vehicle plate, submit Host KYC form (NID + Holding Tax/Allotment Slip + Slot Photo), and Admin 1-click [Approve] button.
* **parking-service (parking_db):** Verified Host creates a parking listing with 1-click browser GPS (navigator.geolocation), toggles spot status (ACTIVE / INACTIVE), and Driver searches by Area dropdown + Vehicle Type in a clean Card Feed.
* **booking-service (booking_db):** Driver books a spot for a selected time window; service calls parking-service via OpenFeign to calculate price (Hours x Rate), saves booking, and unlocks Google Maps iframe preview & URL navigation redirect.
* **Basic Dashboard Summary Cards (SUM / COUNT):** Host Dashboard shows Active Spots (COUNT from parking_db), Total Bookings (COUNT from booking_db), and Total Earnings BDT (SUM from booking_db). Admin Portal shows Pending KYC Requests, Verified Hosts (COUNT from user_db), and Total Active Listings (COUNT from parking_db).

STAGE 2: Geospatial "Wow Factor" Upgrades

* **"Find Near Me" Proximity Search (Haversine SQL):** Add a single @Query in parking-service using the Haversine distance formula so drivers can find spots within 2 km of their current GPS coordinate and see "0.4 km away" on every card.
* **Airbnb-Style Split Screen (Card Feed + Leaflet Clustered Map):** Render free Leaflet.js + OpenStreetMap alongside the Card Feed with Leaflet.markercluster so multiple spots in the same building automatically group into a (3) cluster pin that fans out when clicked.

STAGE 3: Minimal, High-Visual Polish for Final Defense

* **Simple Double-Booking Prevention:**A single SQL COUNT(*) check in booking-service ensuring no existing CONFIRMED booking overlaps the requested start_time and end_time.
* **Printable Digital Parking Pass:** A clean ticket screen showing the Booking ID, Driver Vehicle Plate Number (for building guard verification at the gate), Host address/phone, time window, and a simulated "Paid via Mock Wallet" badge.

5. Mapping to University Defense Phases

Defense Phase	What You Present / Demo	Why It Wins Marks
Phase 1: Title Defense (Current)	Present the Microservices + Database-per-Service Architecture, Isolated ERDs, Dual-Role KYC concept, and the 3-Stage Roadmap.	Shows true microservices isolation (user_db, parking_db, booking_db) + Keycloak OAuth2 + OpenFeign + Geospatial clustering.
Phase 2: Pre-Defense (Mid-term)	Demo Stage 1 (100% Working Microservices MVP + Dashboard Summary Cards) + start of Stage 2.	Complete working flow from Keycloak login -> Host KYC approval -> Listing creation -> Booking & live dashboard counters.
Phase 3: Final Defense (End-term)	Demo Stage 1 + Stage 2 (Haversine + Clustered Map) + Stage 3 (Overlap Prevention & Digital Pass).	Visual split-screen map clustering + live proximity search + complete microservices demonstration.